High Performance Computing

GPU Programming with CUDA

Using Google Colab

A local NVIDIA GPU is **not required** for this exercise. All CUDA programs can be compiled and executed using a GPU provided through Google Colab.

Step 1: Create a Colab Notebook

Open Google Colab in your web browser and create a new notebook.

Save the notebook so that your work is not lost if the runtime disconnects.

Step 2: Enable a GPU

In the Colab menu select:

Runtime → Change runtime type → Hardware accelerator → GPU

Connect to the runtime.

GPU availability depends on Colab resources. The exact GPU model assigned to your notebook may be different for different students.

Step 3: Check the GPU

Create a code cell and run:

```
!nvidia-smi
```

You should see information about the NVIDIA GPU assigned to your runtime.

Record the GPU model:

Item	Result
GPU model	
GPU memory	

Step 4: Check the CUDA Compiler

Run:

```
!nvcc --version
```

The `nvcc` command is the NVIDIA CUDA compiler.

Record the CUDA version reported by the system.

Item	Result
CUDA version	

Creating CUDA Files in Colab

CUDA programs normally use the extension:

`.cu`

You can create a CUDA source file directly from a Colab cell using:

`%%writefile program.cu`

For example:

```
%%writefile hello.cu

#include <stdio.h>

int main()
{
    printf("Hello\n");
    return 0;
}
```

Then compile the file in another cell:

```
!nvcc hello.cu -o hello
```

and execute it:

```
!./hello
```

Throughout this exercise, use the same procedure:

1. create a `.cu` source file;
2. compile it using `nvcc`;
3. execute the compiled program;
4. record the requested results.

Objectives

The purpose of this exercise is to become familiar with the basic CUDA programming model and GPU programming. You will distinguish between host and device execution, create and launch CUDA kernels, work with blocks and threads, calculate global thread indices, allocate GPU memory, transfer data between CPU and GPU memory, perform vector operations in parallel, and investigate how the execution configuration affects a CUDA program.

Part 1: Your First CUDA Kernel

Create the file:

`hello.cu`

with the following program:

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void helloKernel()
{
    printf("Hello from the GPU!\n");
}

int main()
{
    printf("Hello from the CPU!\n");

    helloKernel<<<1, 1>>>();

    cudaDeviceSynchronize();

    return 0;
}
```

Compile:

```
!nvcc hello.cu -o hello
```

Run:

```
!./hello
```

Answer the following questions.

1. Which function executes on the CPU?
2. Which function executes on the GPU?
3. What does the `__global__` qualifier mean?
4. What is a CUDA kernel?
5. What does the following syntax represent?

   ```
   helloKernel<<<1, 1>>>();
   ```

6. What do the two values inside `<<<1,1>>>` specify?

Part 2: Blocks and Threads

Modify the kernel:

```
__global__ void helloKernel()
{
    printf("Block %d, Thread %d\n",
           blockIdx.x,
           threadIdx.x);
}
```

Launch the kernel using:

```
helloKernel<<<2, 4>>>();
```

Run the program and complete the table.

Block	Possible Thread IDs
0	
1	

Answer:

1. How many blocks are created?
2. How many threads are created inside each block?
3. How many threads are created in total?
4. What does `blockIdx.x` represent?
5. What does `threadIdx.x` represent?
6. Is the order in which the messages are printed guaranteed?

Part 3: Global Thread Index

Thread numbers repeat inside every block.

For example, with:

```
<<<3, 4>>>
```

every block contains threads numbered:

$$0, \quad 1, \quad 2, \quad 3.$$

Therefore, a global thread index can be calculated as:

$$\text{index} = \texttt{threadIdx.x} + \texttt{blockIdx.x} \times \texttt{blockDim.x}.$$

Modify the kernel:

```
__global__ void showIndex()
{
    int index =
        threadIdx.x +
        blockIdx.x * blockDim.x;

    printf("Block %d, Thread %d, Global index %d\n",
           blockIdx.x,
           threadIdx.x,
           index);
}
```

Launch:

```
showIndex<<<3, 4>>>();
```

Complete the table.

Block	Thread	Global Index
0	0	
0	1	
0	2	
0	3	
1	0	
1	1	
1	2	
1	3	
2	0	
2	1	
2	2	
2	3	

Answer:

1. Why is `threadIdx.x` alone not sufficient for processing a large vector?
2. What is the value of `blockDim.x` in this example?
3. What global index is calculated for thread 2 in block 2?

Part 4: Host and Device Memory

CUDA programs normally contain separate CPU and GPU memory.

Consider the following variables:

```
1 int *A;
2 int *dev_A;
```

Assume that `A` points to CPU memory and `dev_A` points to GPU memory.

Answer:

1. Which variable refers to host memory?
2. Which variable refers to device memory?
3. What CUDA function is normally used to allocate memory on the device?
4. What CUDA function is used to copy data between host and device?
5. What CUDA function is used to release allocated device memory?

Complete the typical CUDA processing sequence:

1. Allocate host data.
2. _____
3. Copy input data from host to device.
4. _____
5. Copy the result from device to host.
6. _____

Part 5: GPU Vector Addition

You will now implement:

$$C_i = A_i + B_i.$$

Use:

$$N = 1,000,000.$$

Each CUDA thread should calculate one element of the result.
Create:

`vector_add.cu`

Start with:

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void vectorAdd(const float *A,
                          const float *B,
                          float *C,
                          int N)
{
    int i =
        threadIdx.x +
        blockIdx.x * blockDim.x;

    if (i < N)
    {
        // TODO:
        // calculate C[i]
    }
}

int main()
{
    const int N = 1000000;

    size_t bytes = N * sizeof(float);

    float *A = (float*)malloc(bytes);
    float *B = (float*)malloc(bytes);
    float *C = (float*)malloc(bytes);

    for (int i = 0; i < N; ++i)
    {
        A[i] = (float)i;
        B[i] = 2.0f * i;
    }

    float *dev_A;
    float *dev_B;
    float *dev_C;

    // TODO:
    // allocate GPU memory

```

```
44
45      // TODO:
46      // copy A and B to GPU memory
47
48
49      int threadsPerBlock = 256;
50
51      int blocks =
52          (N + threadsPerBlock - 1)
53          / threadsPerBlock;
54
55      // TODO:
56      // launch vectorAdd kernel
57
58
59      // Wait until the GPU finishes
60      cudaDeviceSynchronize();
61
62      // TODO:
63      // copy C back to the CPU
64
65
66      printf("C[0] = %f\n", C[0]);
67      printf("C[10] = %f\n", C[10]);
68      printf("C[N-1] = %f\n", C[N-1]);
69
70      // TODO:
71      // free device memory
72
73
74      free(A);
75      free(B);
76      free(C);
77
78      return 0;
79  }
```

Compile:

```
!nvcc vector_add.cu -o vector_add
```

Run:

```
!./vector_add
```

Verify that:

$$C_i = 3i.$$

Therefore:

$$C_{10} = 30.$$

Questions

1. Why is vector addition suitable for GPU execution?
2. Does thread i need the result produced by thread $i - 1$?

3. Why do we use:

```
if (i < N)
```

inside the kernel?

4. Explain the expression:

```
(N + threadsPerBlock - 1) / threadsPerBlock
```

5. If $N = 1000$ and there are 256 threads per block, how many blocks are required?

6. Some launched threads may have an index greater than or equal to N. What should those threads do?

Part 6: Experiment with Block Size

Run the vector addition program using the following numbers of threads per block:

$$32, \quad 64, \quad 128, \quad 256, \quad 512.$$

For every configuration calculate the required number of blocks.
Use:

$$\text{blocks} = \left\lceil \frac{N}{\text{threads per block}} \right\rceil .$$

Complete the table.

Threads per Block	Number of Blocks
32	
64	
128	
256	
512	

Answer:

1. Does changing the number of threads per block change the mathematical result?

2. Does changing the block size change the way the GPU organizes the computation?

3. Why do CUDA programs normally use both multiple blocks and multiple threads per block?

Part 7: Measuring Kernel Execution Time

CUDA provides event objects that can be used to measure execution time on the GPU.
Add:

```
cudaEvent_t start, stop;

cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);

vectorAdd<<<blocks, threadsPerBlock>>>(
```

```
    dev_A, dev_B, dev_C, N);

cudaEventRecord(stop);

cudaEventSynchronize(stop);

float milliseconds = 0.0f;

cudaEventElapsedTime(
    &milliseconds, start, stop);

printf("Kernel time: %f ms\n",
       milliseconds);

cudaEventDestroy(start);
cudaEventDestroy(stop);
```

Run the experiment several times for each block size.
Complete the table using the average execution time.

Threads per Block	Kernel Time (ms)
32	
64	
128	
256	
512	

Answer:

1. Which configuration produced the lowest execution time?
2. Is the fastest configuration necessarily the one with the largest number of threads per block?
3. Did all configurations produce exactly the same output?
4. Why should GPU timing experiments normally be repeated several times?

Part 8: CPU versus GPU

Implement the same operation on the CPU:

```
for (int i = 0; i < N; ++i)
{
    C[i] = A[i] + B[i];
}
```

Use a sufficiently large value of N such that execution time can be measured reliably.
For example, try:

$$N = 10^6, \qquad 10^7.$$

Record approximate execution times.

N	CPU Time	GPU Kernel Time
10^6		
10^7		

Answer:

1. Which implementation is faster for the larger input?
2. Does the GPU implementation also require time to copy data between CPU and GPU memory?
3. Why can comparing only kernel execution time with complete CPU execution time be misleading?
4. For a very small vector, would you always expect the GPU to be faster? Explain.

Part 9: Two-Dimensional Thread Indexing

GPU programs are not limited to one-dimensional data.

Consider a matrix:

$$A \in R^{N \times N}.$$

For a two-dimensional block, a thread can determine its matrix position using:

```
int col =
    blockIdx.x * blockDim.x +
    threadIdx.x;

int row =
    blockIdx.y * blockDim.y +
    threadIdx.y;
```

Suppose:

$$N = 512.$$

Use:

```
dim3 threads(16, 16);

dim3 blocks(
    (N + threads.x - 1) / threads.x,
    (N + threads.y - 1) / threads.y);
```

Answer:

1. How many threads are contained in one 16×16 block?
2. How many blocks are needed in the x direction for $N = 512$?
3. How many blocks are needed in the y direction?
4. How many blocks exist in the complete grid?
5. Why is two-dimensional thread indexing convenient for matrices and images?

Part 10: Matrix Addition

Implement:

$$C_{i,j} = A_{i,j} + B_{i,j}.$$

Use one GPU thread for each matrix element.
Complete the kernel:

```
__global__ void matrixAdd(
    const float *A,
    const float *B,
    float *C,
    int width,
    int height)
{
    int col =
        blockIdx.x * blockDim.x +
        threadIdx.x;

    int row =
        blockIdx.y * blockDim.y +
        threadIdx.y;

    if (row < height && col < width)
    {
        int index = row * width + col;

        // TODO:
        // calculate C[index]
    }
}
```

Use:

$$512 \times 512$$

matrices.
Launch the kernel using:

```
dim3 threads(16, 16);

dim3 blocks(
    (width + threads.x - 1) / threads.x,
    (height + threads.y - 1) / threads.y);

matrixAdd<<<blocks, threads>>>(
    dev_A,
    dev_B,
    dev_C,
    width,
    height);
```

Verify several elements of the resulting matrix.

Questions

1. Which thread calculates one matrix element?
2. Why are both row and column indices required?

3. Why is the matrix stored in a one-dimensional array in this example?
4. What is the purpose of:

```
row * width + col
```

?

Part 11: CUDA Memory Model

For each type of memory, indicate its scope.

Memory	Visible To
Registers	
Local memory	
Shared memory	
Global memory	

Answer:

1. Which memory is visible to all threads in a grid?
2. Which memory is shared by threads belonging to the same block?
3. Can threads in different blocks directly use the same shared-memory array?
4. Why can shared memory be faster than global memory?
5. Why is data reuse important when optimizing GPU programs?

Part 12: Synchronization

Consider several threads belonging to one thread block.
CUDA provides:

```
__syncthreads();
```

Answer:

1. What is the purpose of `__syncthreads()`?
2. Does it synchronize threads belonging to one block or the entire grid?
3. Why might synchronization be required after several threads write data to shared memory?
4. Can we assume that CUDA threads finish in numerical order?
5. Can we assume that CUDA blocks finish in numerical order?

Part 13: Short Discussion

Answer each question briefly.

1. What is the difference between the host and the device?
2. What is a CUDA kernel?
3. What is the difference between a thread, a block, and a grid?
4. What is the purpose of `threadIdx`?
5. What is the purpose of `blockIdx`?
6. What is the purpose of `blockDim`?
7. Why is GPU computing particularly suitable for data-parallel problems?
8. What is the purpose of `cudaMalloc()`?
9. What is the purpose of `cudaMemcpy()`?
10. Why must host-to-device and device-to-host transfers be considered when evaluating GPU performance?
11. Why are thousands of CUDA threads commonly used instead of only a few threads?
12. Why can several different CUDA execution configurations produce the same correct result but different performance?

Optional Challenge: Matrix Multiplication

This part is optional.

Implement the matrix multiplication:

$$C = A \times B.$$

Each GPU thread should calculate one element:

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} B_{k,j}.$$

Use a two-dimensional grid and two-dimensional blocks.

You do **not** need to use shared-memory tiling for this exercise.

Test your implementation using small matrices first, and compare the GPU result against a CPU implementation.

Submission

Submit:

1. your Google Colab notebook (`.ipynb`);
2. a screenshot or saved output of `nvidia-smi` showing the GPU used;
3. `hello.cu`;
4. the completed vector addition program;

5. the completed matrix addition program;
6. the block-size performance table;
7. the CPU/GPU comparison table;
8. answers to all questions.

The submitted Colab notebook should contain the program code and its execution output. All CUDA programs must compile successfully using `nvcc`.